

leogpava:~\$./finops --serve :8080

FinOps Crédito API

Cadastro da operação de crédito, calendário de parcelas e controle de inadimplência.
Esta é a referência para integrar com o módulo de FinOps da esteira.

BASE `http://localhost:8080` FORMATO `JSON` · `UTF-8` AUTH `nenhuma` CORS `liberado (GET, POST)`

CONVENÇÕES

Tipos e formatos
Formato de erro

ENDPOINTS

`POST` `/operacoes`
`GET` `/operacoes`
`POST` `.../parcelas/importar`

`GET` `.../parcelas`
`POST` `.../pagamento`

INTEGRAÇÃO

Situação da parcela
Dependência: API de Juros
Fluxo ponta a ponta
Rodando local

Tipos e formatos

Três convenções valem para todos os endpoints e evitam a maior parte dos erros de integração:

- **Datas** são sempre texto no formato `YYYY-MM-DD`. Nunca timestamp, nunca com fuso.
- **Valores monetários** são números JSON com 2 casas decimais em reais — `916.80`, não `"916.80"` nem centavos inteiros.
- **A taxa de juros** é mensal em percentual: `1.5` significa 1,5% a.m.

O `operacaoID` é o identificador de negócio da operação (ex.: `OP-0001`), único, e é por ele que todos os endpoints de parcela são chamados. O `id` em UUID que aparece nas respostas é interno do banco.

Formato de erro

Toda falha devolve o mesmo envelope, com a mensagem em português pronta para exibir ao usuário final:

```
{ "erro": "operação não encontrada" }
```

CÓDIGO	USO
--------	-----

200	Leitura ou baixa concluída
-----	----------------------------

201	Operação cadastrada ou parcelas importadas
-----	--

400	Qualquer erro de validação ou de regra de negócio — inclusive recurso inexistente
-----	--

500	Falha inesperada ao consultar o banco
-----	---------------------------------------

Não usamos 404 nem 409. Operação inexistente, parcela já paga e reimportação vêm todos como 400 — diferencie pela mensagem em erro, que é estável.

Cadastrar operação

POST /operacoes

Registra o cabeçalho da operação de crédito. É o primeiro passo — sem ele nenhum endpoint de parcela funciona.

CORPO

CAMPO	TIPO	REGRA
operacaoID	string	Obrigatório, único
valorFinanciado	number	Maior que zero
quantidadeParcelas	integer	Maior que zero
taxaJuros	number	Maior ou igual a zero, % ao mês
primeiroVencimento	string	YYYY-MM-DD
sistemaAmortizacao	string	PRICE ou SAC (aceita minúsculo)

```
// requisição
{
  "operacaoID" : "OP-0001",
  "valorFinanciado" : 10000.00,
  "quantidadeParcelas" : 12,
  "taxaJuros" : 1.5,
  "primeiroVencimento" : "2026-10-20",
  "sistemaAmortizacao" : "PRICE"
}
```

RESPOSTA 201

```
{
  "id" : "550e8400-e29b-41d4-a716-446655440000",
  "operacaoID" : "OP-0001",
  "valorFinanciado" : 10000,
  "quantidadeParcelas" : 12,
  "taxaJuros" : 1.5,
  "primeiroVencimento" : "2026-10-20",
  "sistemaAmortizacao" : "PRICE"
}
```

ERROS 400

- corpo da requisição inválido

- `operacaoID` não pode ser vazio
- `valorFinanciado` deve ser maior que zero
- `quantidadeParcelas` deve ser maior que zero
- `taxaJuros` deve ser maior ou igual a zero
- `primeiroVencimento` não pode ser vazio · `primeiroVencimento` deve estar no formato `YYYY-MM-DD`
- `sistemaAmortizacao` deve ser `PRICE` ou `SAC`

Listar operações

GET `/operacoes`

Devolve `200` com um array de operações — mesmo objeto da resposta anterior — ordenado por `primeiroVencimento` e depois `operacaoID`. Sem nada cadastrado, devolve `[]`, nunca `null`.

Importar o calendário de parcelas

POST `/operacoes/{operacaoID}/parcelas/importar`

Este é o ponto de integração com o grupo de Juros. Não tem corpo. Ao ser chamado, o FinOps consulta a API de Juros, valida o calendário recebido e grava as parcelas em uma única transação — ou grava todas, ou nenhuma.

Responde `201` com o array de parcelas gravadas, no mesmo formato do endpoint seguinte.

ERROS 400

MENSAGEM	QUANDO
operação não encontrada	<code>operacaoID</code> não cadastrado
operação já possui parcelas importadas	O import é uma única vez por operação
calendário recebido não bate com a <code>quantidadeParcelas</code> da operação	A API de Juros devolveu um número de parcelas diferente do cadastrado
não foi possível consultar a api de juros	Timeout de 10s ou API fora do ar
api de juros retornou status 500	A API de Juros respondeu diferente de <code>200</code>
resposta da api de juros em formato inválido	JSON malformatado
api de juros não retornou parcelas	Lista vazia
api de juros retornou parcela com numero inválido	<code>numero</code> menor ou igual a zero
api de juros retornou vencimento fora do formato <code>YYYY-MM-DD</code>	Data inválida

JUROS_API_URL não configurada

Variável de ambiente ausente no FinOps

Listar parcelas

GET /operacoes/{operacaoID}/parcelas

Devolve `200` com as parcelas ordenadas por `numero`. Antes do import, devolve `[]`.

Operação inexistente devolve `400` operação não encontrada.

```
[
  {
    "id" : "9f1c2b7a-3d4e-4f5a-8b6c-7d8e9f0a1b2c",
    "operacaoID" : "OP-0001",
    "numero" : 1,
    "vencimento" : "2026-10-20",
    "valorParcela" : 916.8,
    "amortizacao" : 766.8,
    "juros" : 150,
    "saldoDevedor" : 9233.2,
    "dataPagamento" : "2026-10-20",
    "status" : "PAGA",
    "diasAtraso" : 0
  },
  {
    "id" : "1a2b3c4d-5e6f-4a7b-8c9d-0e1f2a3b4c5d",
    "operacaoID" : "OP-0001",
    "numero" : 2,
    "vencimento" : "2026-11-20",
    "valorParcela" : 916.8,
    "amortizacao" : 778.3,
    "juros" : 138.5,
    "saldoDevedor" : 8454.9,
    "dataPagamento" : "",
    "status" : "ATRASADA",
    "diasAtraso" : 47
  }
]
```

Dar baixa em uma parcela

POST /operacoes/{operacaoID}/parcelas/{numero}/pagamento

Registra o pagamento da parcela. Só existe baixa total — não há pagamento parcial nesta versão.

CORPO — OPCIONAL

Sem corpo, a baixa é registrada com a data de hoje.

```
{ "dataPagamento" : "2026-10-20" }
```

RESPOSTA 200

A parcela atualizada, já com `status` em `PAGA` e `diasAtraso` zerado.

A baixa não é reversível e não sobrescreve. Chamar duas vezes na mesma parcela devolve parcela não encontrada ou já está paga e preserva o primeiro pagamento — é seguro reprocessar uma fila sem duplicar.

ERROS 400

- `numero da parcela inválido` — o valor na URL não é número
- `numero da parcela deve ser maior que zero`
- `dataPagamento` deve estar no formato YYYY-MM-DD
- `operação não encontrada`
- `parcela não encontrada ou já está paga`

Situação da parcela

`status` e `diasAtraso` **não são armazenados** — são calculados a cada leitura, comparando o vencimento com a data de hoje. Não é preciso rodar nenhuma rotina noturna para uma parcela virar inadimplente: ela vira sozinha na virada do dia.

STATUS	SIGNIFICADO	DIASATRASSO
PAGA	Tem <code>dataPagamento</code> preenchida	0
EM_ABERTO	Em aberto e ainda dentro do prazo	0
ATRASADA	Em aberto e o vencimento já passou	dias corridos de atraso

`dataPagamento` vem como string vazia (`"`) enquanto a parcela não foi paga — nunca `null`.

Dependência: API do grupo de Juros

Para o import funcionar, o FinOps precisa da variável `JUROS_API_URL` apontando para a API de Juros, que deve expor:

```
GET {JUROS_API_URL}/calendario
```

QUERY PARAMS QUE ENVIAMOS

PARAM	EXEMPLO
<code>valorFinanciado</code>	10000.00
<code>quantidadeParcelas</code>	12
<code>taxaJuros</code>	1.5000

PARAM	EXEMPLO
primeiroVencimento	2026-10-20
sistemaAmortizacao	PRICE

RESPOSTA ESPERADA – 200

```
{
  "parcelas" : [
    { "numero" : 1, "vencimento" : "2026-10-20", "valorParcela" : 916.80,
      "amortizacao" : 766.80, "juros" : 150.00, "saldoDevedor" : 9233.20 },
    { "numero" : 2, "vencimento" : "2026-11-20", "valorParcela" : 916.80,
      "amortizacao" : 778.30, "juros" : 138.50, "saldoDevedor" : 8454.90 }
  ]
}
```

O QUE VALIDAMOS NA ENTRADA

- A lista não pode ser vazia.
- `numero` tem que ser maior que zero.
- `vencimento` tem que estar em `YYYY-MM-DD`.
- A quantidade de itens tem que ser igual ao `quantidadeParcelas` da operação.
- Timeout de 10 segundos. Campos extras no JSON são ignorados.

Fluxo ponta a ponta

A ordem importa — cada passo depende do anterior:

1	POST /operacoes Cadastra OP-0001 com valor, prazo, taxa e sistema de amortização.
2	POST /operacoes/OP-0001/parcelas/importar O FinOps chama a API de Juros e grava as 12 parcelas em transação.
3	GET /operacoes/OP-0001/parcelas Acompanha status e dias de atraso, recalculados a cada leitura.
4	POST /operacoes/OP-0001/parcelas/1/pagamento Dá baixa na parcela 1 e ela passa a PAGA.

CURL

```
# 1. cadastrar
curl -X POST http://localhost:8080/operacoes \
  -H 'Content-Type: application/json' \
  -d @examples/inboundDecisionPayload.json

# 2. importar o calendário do grupo de Juros
curl -X POST http://localhost:8080/operacoes/OP-0001/parcelas/importar

# 3. acompanhar
curl http://localhost:8080/operacoes/OP-0001/parcelas

# 4. dar baixa na parcela 1
curl -X POST http://localhost:8080/operacoes/OP-0001/parcelas/1/pagamento \
  -H 'Content-Type: application/json' \
  -d '{"dataPagamento":"2026-10-20"}'
```

Rodando local

```
# 1. Postgres com as tabelas financiamento e parcela
psql $DATABASE_URL -f examples/table.sql

# 2. .env na raiz do projeto
DATABASE_URL=postgres://usuario:senha@localhost:5432/finops?sslmode=disable
JUROS_API_URL=http://localhost:8081

# 3. sobe na :8080, com o front de demonstração em /
go run .
```

Enquanto a API de Juros não estiver de pé, qualquer servidor estático servindo o payload de calendário em `/calendario` na porta configurada resolve para testar o import ponta a ponta.

Fora do escopo desta versão: carteira consolidada e aging por faixa de atraso, multa e juros de mora, PDD, dados do cliente, pagamento parcial, estorno de baixa e autenticação.